

Annex A

Technical Architecture Deep-Dive

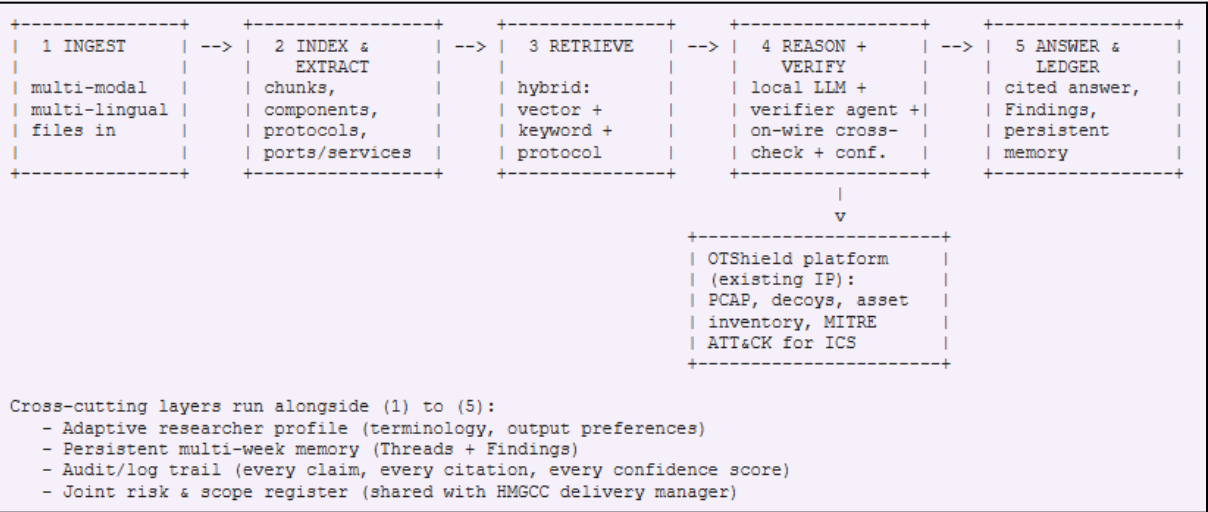
Supporting document to OTShield's HMGCC Co-Creation Proposal: Smart Personal Assistant for Security Researchers

This annex is **optional reference material** that provides supporting evidence for claims already made and assessable within the six-page main proposal (Section 2 in particular). Per HMGCC Q&A (Q6), all assessable information is contained in the main proposal itself; this annex is provided only to give the reviewer faster verification of those claims if desired. It contains the system architecture diagram, the end-to-end data flow, module-by-module specifications, and the new IP register for Phase 1. Reading this annex is not required for evaluation.

A.1 System architecture: five cooperating modules

The Phase 1 tool is a locally-hosted web application that runs entirely on the researcher's laptop. The backend, local LLM (Ollama-class), embedding store, indices and structured-data layer all run as local processes on the same machine; the researcher interacts with the tool through their browser at localhost. There are no external network calls. It is built around five cooperating modules wrapped by a research-first UI (Bundles, Library, Summary, Inventory, Threads, Findings, Vulns). All data, models, indices and conversation history stay on the local machine: nothing leaves the laptop.

Why a locally-hosted web application rather than a native desktop binary. A localhost-served web UI lets us reuse OTShield's existing, production-tested Research Mode and SOC Mode interfaces without rewriting them as native Windows or Linux desktop apps; gives the researcher a familiar, modern UI without sacrificing the air-gap; supports the same offline browser interface across Windows and Linux laptops with one codebase; and is the same delivery model used by air-gap-friendly tools the researcher already knows (e.g. Jupyter, Wireshark / Stratoshark, Splunk Free, Grafana). The application is packaged with a signed installer that brings up the local services on first run; once installed, the laptop never needs an internet connection.



A.2 End-to-end flow, by module

Module 1, Ingest (multi-modal, multilingual)

The researcher drops files into a Bundle: vendor PDFs, datasheets, schematics, draw.io diagrams, hand-drawn notes, photographs, Word / Excel / PowerPoint documents, forum dumps and screenshots (per HMGCC Q21, binary firmware and source code are out of scope for HMGCC test data; the assistant works from documentation about hardware / software systems, not from compiled binaries or source). Module 1 routes each input through the right extractor:

- **OCR pipeline** for scanned and printed documents, tuned to ICS vocabulary (PLC model numbers, register addresses, protocol names).
- **Handwriting recognition** for engineer field notes and annotations.
- **Image captioning** for schematics, board photos and tear-down images.
- **Language detection** per chunk; non-English material is routed through the offline neural translator (Phase 1 deliverable; HMGCC priority languages DE, JA, FR, ZH, with extension headroom for additional languages).
- **Code parsers** for vendor binaries, source and configuration files.

Module 2, Index & extract

Content is chunked, embedded and stored locally. Two parallel pipelines run:

- **Vector index:** language-agnostic ICS-tuned embeddings; non-English chunks indexed in both original and translated form for traceability.
- **Deep-extract LLM passes:** populate structured tables (Inventory, Ports & Services, Components, Protocols). Today the deep-extract pass is LLM-driven and protocol-agnostic (catches references like "uses CoAP over DTLS" that regex misses). Phase 1 layers on protocol-specific extractors for Modbus, S7Comm, EtherNet/IP, OPC UA, DNP3 and IEC 60870-5-104 that recognise function codes, register addresses, object identifiers and protocol-specific terminology, with corresponding network-side dissectors evolved from the existing Modbus and S7Comm DPI rules.

The output is not just a vector store: it is a structured knowledge base of the device's architecture, ready for the Inventory and Ports & Services views in the UI.

Module 3, Retrieve

When the researcher asks a question, Module 3 runs hybrid retrieval:

- Vector similarity over the embedded chunks.
- Keyword filters over the structured tables (e.g. "give me everything about port 502").
- Protocol-aware filters (e.g. "Modbus function codes only").
- Cross-lingual expansion: an English query also retrieves matching German, French, Chinese passages.

Result: a ranked list of passages plus the structured-table evidence to support each answer.

Module 4, Reason & verify (the differentiator)

This is where OTShield's existing platform creates a step-change beyond standard RAG. The local LLM drafts an initial answer; an independent **verifier agent** then runs three parallel checks:

1. **Corpus contradiction check:** independently re-queries the corpus to confirm or refute the draft answer.
2. **Trusted-source cross-check:** compares against high-confidence sources (manufacturer documentation, academic publications, advisory snapshots) listed in the trusted-source register that we will curate jointly with HMGCC during the kick-off.
3. **On-wire cross-check (unique to OTShield):** compares the documented behaviour against

actual PCAP traffic, decoy interactions and live asset telemetry from OTShield's existing platform. A claim like "the device exposes Modbus on port 502" can be confirmed (or refuted) against real traffic.

Each claim emerges with a calibrated 0 to 1 confidence score and a 'needs more sources' flag if evidence is thin.

Module 5, Answer & ledger

The verified answer is rendered in the Threads UI with inline citations. The researcher can:

- Promote any answer (or sub-claim) to a permanent **Finding**, which is auditable and reproducible.
- Open the **Vulns** view to track observations and 'needs more sources' items across the whole bundle.
- Continue the conversation across days or weeks; the assistant remembers prior questions, answers and the researcher's preferred output format.

A.3 Innovation layers mapped to modules

Layer	Module(s)	Beyond standard RAG
L1: Multi-modal ICS-aware & multilingual ingest	1, 2	OCR + handwriting tuned to ICS vocabulary; protocol-aware extractors that auto-build Inventory and Ports & Services. Multilingual extension in Phase 1: language-agnostic core, offline neural translation for HMGCC priority languages (DE, JA, FR, ZH), bilingual technical-term dictionary curated with HMGCC, and cross-language retrieval so an English query answers from a German datasheet.
L2: Agentic verification + on-wire cross-check	4	Independent verifier agent runs three parallel checks (corpus, trusted sources, real PCAP/decoy traffic). Hallucinations and unsupported claims are flagged before the answer is published. The on-wire cross-check is unique to OTShield and only possible because the platform already has a live PCAP / decoy / asset stack running today on an internet-exposed decoy fabric. Verifier methodology is shaped by Microsoft PyRIT-class AI red-teaming techniques, applied by Jon Medvenics (ChCSP) on the project (see Annex C).
L3: Confidence scoring & evidence ledger	4, 5	Every claim ships with a numeric 0 to 1 confidence + source list + 'needs more sources' flag. The Findings ledger is auditable, reproducible and exportable: defensible in a security review or audit.
L4: Adaptive profile + persistent memory	cross-cutting	Threads + Findings persist for weeks without re-prompting. The system learns the researcher's terminology and preferred output style (table, prose, schematic-style), and proactively surfaces related items from prior bundles. Local only: no telemetry leaves the laptop.

A.4 Updates without internet (the air-gap-safe feed)

HMGCC's challenge requires a 100 % offline tool. The desirable requirement asks for a mechanism to keep the tool current despite the air gap. Our approach:

1. OTShield's **commercial** threat-intel platform (running outside the air-gapped environment, on OTShield infrastructure) continuously collects ICS attack data from internet-exposed decoys.
2. The collected intel is curated, signed and packaged as **update bundles** (new attacker TTPs, new IOCs, refreshed protocol-aware extractors, refreshed trusted-source snapshots).
3. The **air-gapped researcher tool** ingests these bundles via removable media or a local

watch-folder. The laptop never connects to the internet, but its corpus and detection signatures stay current.

This is the same pattern used by air-gap-safe AV signature distribution and offline mapping data: trusted authority publishes signed bundles, air-gapped client validates and ingests.

Architectural separation: the Phase 1 tool delivered to HMGCC contains *only* the offline components. The internet-facing decoy fabric and intel pipeline are entirely separate, on Safetech Global's own infrastructure. HMGCC receives bundles, never an internet connection.

A.5 New IP generated in Phase 1 (in detail)

New IP module	Description
Agentic verification stack with on-wire cross-check	Verifier agent orchestration, three-pass cross-checking, contradiction-detection algorithms, bridge to the existing OTShield PCAP / decoy data store.
Calibrated ICS confidence model	0 to 1 confidence calibrated against ICS-domain ground truth (using HMGCC test corpus and OTShield's internal ICS data set), with a 'needs more sources' threshold.
Adaptive researcher-profile module	Per-researcher terminology and output-format learning, with proactive surfacing of related Findings from prior bundles. Local-only model state.
Multilingual / cross-lingual layer	On-device language detection, offline neural translation for HMGCC priority languages (DE, JA, FR, ZH), bilingual technical-term dictionary curated with HMGCC, cross-lingual retrieval expansion.
Long-term memory schema	Cross-bundle, cross-thread, multi-week conversation memory, including 'continue from last week' workflows.
Signed update bundle pipeline	(Phase 1 design, Phase 2 build) bundle format, signature scheme, watch-folder ingest path, conflict resolution. Designed in Phase 1 for delivery in Phase 2.

All IP listed above is owned by OTShield. The Authority is granted a non-exclusive, royalty-free licence under the HMGCC Co-Creation terms and conditions.

A.6 Existing IP brought to the project (no charge to HMGCC)

OTShield Research Workbench: Workspace, Library, Summary, Inventory, Ports & Services, Threads, Findings, Vulns.

- Deep-extract pipeline (PDF text via Apache PDFBox + plain-text fallback for markdown / text / CSV) plus an LLM-driven inventory deep-extractor; dedicated network dissectors for Modbus and S7Comm with anomaly rules; port-based identification for DNP3, OPC UA, EtherNet/IP and IEC 60870-5-104 (their dedicated dissectors are extended in Phase 1 against HMGCC test data).
- Java + Spring Boot backend serving the Research and SOC modes; offline Ollama integration running **Llama 3.2 1B Instruct Q4_K_M** as the chat model and **nomic-embed-text** as the embedding model. Phase 1 upgrades the chat model to Gemma 4 (see A.7).
- In-memory vector store (ConcurrentHashMap + brute-force cosine similarity) for the existing prototype-scale corpus. Phase 1 adds ChromaDB as the persistent, scalable replacement for production-scale corpora.
- OTShield SOC platform: decoy fabric, attacker behaviour analytics, asset inventory,

- Purdue-aware network topology, MITRE ATT&CK for ICS coverage.
- Live ICS threat-intelligence feed from an internet-exposed decoy fabric operated by Safetech Global (see Annex B for evidence).

A.7 Technology stack: today vs. Phase 1 upgrade

The Research Mode that HMGCC's Phase 1 funding extends is a working prototype today (TRL 3 to 4) running a deliberately minimal, production-stable stack; the SOC-side decoy fabric and threat-intel feed it draws on sit at TRL 5 (validated against live attackers on Safetech Global's internet-exposed deployment, see Annex B). Phase 1 takes the Research Mode stack to TRL 6 on a 3-tier deployment architecture: a Tier 1 edge laptop (4 to 8 GB VRAM) as the primary deliverable, an optional Tier 2 escalation path to HMGCC's hosted OpenAI-style endpoint for higher-accuracy verifier passes, and a Tier 3 OTShield internal dev workstation used for benchmarking and calibration only (not part of the deliverable). We list the today and Phase 1 states side by side so the HMGCC review panel can see exactly what is operational today and exactly what the £60K Phase 1 funding builds.

*Note on the chat model vs. embedding model distinction below: these are two distinct layers in the RAG architecture. The **chat model** generates the natural-language answer the researcher reads; the **embedding model** encodes both the user's question and every indexed document chunk into vectors so the retriever can rank passages by semantic similarity before the chat model is even invoked. Both layers are upgraded in Phase 1, but for different reasons (chat model: better ICS-jargon accuracy and agentic capability; embedding model: native multilingual coverage for the desirable requirement).*

Component	Today (operational, in production code)	Phase 1 upgrade (delivered by week 12)
Backend runtime	Java 21 + Spring Boot. Cross-platform, mature, well-suited to UK-government supplier assurance.	Retained. Phase 1 work extends the same Spring Boot service tree; no rewrite.
Local LLM serving	Ollama (localhost:11434), Apache 2.0, packaged for offline use.	Retained. New Phase 1 models are pulled into the same Ollama daemon.
Chat model	Llama 3.2 1B Instruct Q4_K_M. Deliberately small, runs on any laptop CPU, sufficient for prototype-stage research summarisation; 2 048-token context.	3-tier deployment. Tier 1 (edge laptop, 4 to 8 GB VRAM, primary deliverable): Gemma 4 9B Q4_K_M for 4 GB-class laptops, Gemma 4 13B Q4_K_M for 8 GB-class laptops; both Apache 2.0 open-weights, fully air-gapped, no closed-source model dependency. Llama 3.2 1B retained as CPU-only / ablation fallback. Tier 2 (optional scale-up to HMGCC's OpenAI-style endpoint): any open-weights model of HMGCC's choice (e.g. Llama 3.3 70B, Qwen 2.5 72B, Gemma 4 27B). Used by the L2 verifier for higher-accuracy cross-checking, and by an automatic escalation path when Tier 1 confidence falls below threshold. Air-gap preserved (HMGCC-hosted infrastructure). Tier 3 (OTShield internal dev workstation, RTX 5090 32 GB, not part of HMGCC deliverable): Gemma 4 26B MoE / 31B Dense Q5_K_M for benchmark-driven model selection, conformal-prediction calibration and ablation studies. Broad multilingual coverage on Tiers 1-3, complemented by NLLB-200 (200+

Component	Today (operational, in production code)	Phase 1 upgrade (delivered by week 12)
		languages) for translation ; HMGCC priority languages (DE, JA, FR, ZH) validated and benchmarked in Phase 1.
Embedding model	nomic-embed-text via Ollama, 768-dim, English-centric.	BGE-M3 (BAAI, multilingual, MIT) added alongside nomic-embed-text: unified dense / sparse / multi-vector retrieval, 100+ languages, 8K input. Multilingual desirable requirement runs on this layer.
Vector store	InMemoryVectorStore (ConcurrentHashMap + brute-force cosine similarity, < 1 ms search at ≤ 500 chunks). Sufficient for prototype-scale corpora; rebuilt on each app start.	ChromaDB (Apache 2.0, embedded mode, fully offline-capable) as persistent, scalable replacement. Existing < 500-chunk InMemoryVectorStore continues to serve as a fast in-process cache. Hybrid sparse + dense retrieval enabled.
Retrieval pipeline	Top-K = 4, min cosine relevance = 0.18, single-modal (vector only).	Hybrid retrieval: dense (BGE-M3 + nomic), sparse (BM25 over Chroma), and ICS-protocol-aware filters over the structured Inventory / Ports / Services tables. Cross-lingual expansion at query time.
Verifier agent	None today. Answers are returned with retrieved-chunk citations only.	New IP, Phase 1. Independent agentic verifier orchestrated with LangGraph (MIT). Three parallel checks: corpus re-query, trusted-source cross-check, on-wire cross-check (against OTShield PCAP / decoy data, see Annex B). Adversarial validation by Jon Medvenics with PyRIT (Microsoft, MIT).
Confidence scoring	HIGH / MEDIUM / LOW confidence label and binary "needs more sources" flag emitted by the assistant on every answer, plus per-chunk cosine relevance from the retriever (not user-facing as a calibrated 0 to 1 confidence yet).	New IP, Phase 1. Calibrated 0 to 1 confidence per claim using conformal prediction as the primary technique; Platt scaling and isotonic regression evaluated as baselines on the HMGCC calibration set. "Needs more sources" flag below configurable threshold.
OCR / handwriting / vision	PDF text extraction (PDFBox via Spring) and basic image handling.	Tesseract 5 (Apache 2.0) primary OCR; PaddleOCR as a parallel pass for layout-heavy schematics; TrOCR (Microsoft, MIT) for handwriting, prioritising annotations overlaid on existing documents (e.g. handwritten notes on a printed schematic) over standalone notebook pages, per HMGCC Q22; LayoutLMv3 for structured page-layout extraction. Vision captions via Gemma 4's native multimodal capability.
Multilingual translation	Output translation in production for 5 languages (EN, TR, DE, FR, ES) via the local Ollama chat-model translator	NLLB-200 distilled-600M (Meta) packaged offline upgrades the chat-model translator and prioritises HMGCC's stated languages

Component	Today (operational, in production code)	Phase 1 upgrade (delivered by week 12)
	with a domain-tuned prompt that preserves citations, CVE ids, port numbers, IPs and ICS acronyms; LRU-cached per session (see A.11).	(DE, JA, FR, ZH); bilingual technical-term dictionary curated jointly with HMGCC; cross-lingual retrieval over BGE-M3.
Frontend	React (TypeScript) served from localhost; existing Research Mode + SOC Mode UIs.	Retained. Phase 1 adds the verifier-output rendering, confidence visualisation and bias-flag UI components.
Air-gap update bundle	Manual model pulls via Ollama (offline pre-pulled into project environment).	Signed tar.zst bundles with cosign (sigstore, MIT) signatures. Watch-folder ingest verifies signature before unpack. Designed in Phase 1, productionised in Phase 2.

All Phase 1 dependencies are pinned to specific versions in the Phase 1 SBOM (CycloneDX), which is delivered with the build (see Annex C, C.4). At runtime the air-gapped tool makes no internet calls; model weights, indices and toolchain are all part of the offline install package.

Why this honest "today vs. Phase 1" framing matters. The HMGCC challenge sets a TRL 6 target. A TRL 6 deliverable cannot be assessed without knowing the starting point. By stating exactly what is operational today (Java + Ollama + Llama 3.2 1B + nomic-embed-text + InMemoryVectorStore) and exactly what the Phase 1 funding adds, we give HMGCC's review panel an unambiguous yardstick for measuring TRL movement at week 12.

A.8 Performance targets and benchmarks (TRL 6 evidence)

Phase 1 commits to specific, measurable success criteria, not just qualitative outcomes. The benchmark harness is defined in Sprint 1 with HMGCC and run repeatedly through Sprint 2 and Sprint 3. Targets are deliberately set at "ambitious but achievable" levels for a 12-week project against an HMGCC-supplied corpus.

Metric	Target	Definition
Hallucination rate (verifier-flagged or source-contradicted claims, per 100 answers)	$\leq 5 \%$	Random sample of 200 generated claims; manually adjudicated by Jon and the HMGCC reviewer pool.
Confidence calibration ECE (Expected Calibration Error)	≤ 0.10	Computed against a held-out calibration set with 10 confidence buckets. Conformal prediction baseline.
Retrieval precision@5 on ICS protocol queries	≥ 0.80	HMGCC-provided test queries against the project corpus; relevance judged at chunk level.
Cross-lingual retrieval recall@10 (English query, non-English source)	≥ 0.65	Across HMGCC priority languages (DE, JA, FR, ZH); uses synthetic + HMGCC-supplied bilingual evaluation pairs.

Metric	Target	Definition
Median end-to-end answer latency, Tier 1 edge laptop (4 to 8 GB VRAM, fully air-gapped)	≤ 30 s	From query submission to grounded, cited answer rendered, including local L2 verifier pass. Measured on Gemma 4 9B Q4_K_M (4 GB-class laptop) and Gemma 4 13B Q4_K_M (8 GB-class laptop).
End-to-end answer latency, Tier 2 HMGCC endpoint (verifier escalation path)	≤ 8 s	Same flow when the L2 verifier escalates a low-confidence Tier 1 answer to HMGCC's hosted OpenAI-style endpoint (e.g. Llama 3.3 70B). Air-gap preserved through HMGCC's own infrastructure boundary.
OCR + handwriting accuracy on the schematic-and-notes test set	≥ 0.92 character accuracy	Standard CER on a HMGCC-representative set drawn from Sprint 1 deliverable.
"Needs more sources" precision (when the system flags low confidence, was it right to do so?)	≥ 0.85	Avoids the failure mode where the system over-confidently asserts thin claims.

The TRL 6 evidence pack delivered at week 12 includes: the benchmark harness (reproducible by HMGCC), the raw measurement results, calibration plots and confusion matrices, an adjudication log, and a signed-off methodology document.

A.9 Verifier agent design (the L2 differentiator)

The verifier agent is the technical heart of how this proposal differs from off-the-shelf RAG. It is implemented as a state machine with explicit pass / flag / escalate transitions, rather than as a single LLM-judges-itself prompt.

Per-claim verification protocol:

1. **Claim extraction.** Module 4 splits the draft answer into atomic claims (entity, predicate, value, citation slot).
2. **Independent corpus re-query.** For each claim, the verifier issues an independent retrieval (different prompt phrasing, different top-k) against the same corpus and measures evidence convergence.
3. **Trusted-source cross-check.** Claims relating to vendor specifications, protocol behaviour, or component data are cross-checked against the trusted-source register curated with HMGCC in S1.
4. **On-wire cross-check (unique to OTShield).** For claims about protocol behaviour, function codes, port numbers and runtime characteristics, the verifier consults OTShield's PCAP / decoy / asset evidence base. A claim like "Modbus function code 0x06 writes a single register" is verified against actual on-wire traffic captured from production decoys (see Annex B).
5. **Contradiction detection.** If two retrieval passes return mutually contradictory passages, the claim is flagged as *contested* and rendered to the researcher with both alternatives, not collapsed into a single answer.
6. **Confidence assignment.** A calibrated 0 to 1 confidence score is computed using conformal prediction over the agreement signals from steps 2 to 5.
7. **Threshold gate.** Claims below a configurable confidence threshold (default 0.6, HMGCC-tuneable) are rendered with the *"needs more sources"* flag and an automatically generated suggestion of which source classes are missing.

Adversarial validation. Jon Medvenics applies **PyRIT-class** AI red-team probes against the verifier in Sprint 2 and Sprint 3 : prompt injection through ingested documents, conflicting-evidence injection, citation tampering, jailbreak via researcher-impersonating prompts, and language-switching attacks. Every successful attack is converted to a regression test.

A.10 Multi-modal ingest pipeline (the L1 detail)

Multi-modal ingest runs as a directed pipeline; each stage is independently testable and can be re-run on existing bundles without re-ingesting raw files.

- **Stage 1, Format detection.** File type is identified from magic bytes, not extension. Routing is deterministic. The detector recognises every format in HMGCC's test-data list: PDF, Word (.docx), Excel (.xlsx), PowerPoint (.pptx), plain-text (.txt, .md, .csv), JPEG, PNG, BMP, TIFF, draw.io (.drawio XML), and scanned / photographed images. An optional desirable extension supports database access (CSV, SQL dump, XML, JDBC / ODBC) as a queryable source.
- **Stage 2, Extraction.** PDFs through Apache PDFBox (text and basic structure) plus Tesseract / PaddleOCR fallback for scanned pages. Word, Excel, PowerPoint through **Apache POI** (Apache 2.0, Java-native, fully offline). draw.io diagrams parsed as XML and converted to a structured node-edge graph (text labels, connections and annotations preserved). Images through Gemma 4 vision + LayoutLMv3. Handwriting (both standalone notes and overlaid annotations on schematics) through TrOCR. Forum dumps and screenshots through a custom HTML / JSON normaliser plus OCR.
- **Stage 2.1, Architectural-insight extraction and knowledge graph (per HMGCC Q14 + Q21 + Q34 + Q35).** The LLM reads the ingested **documentation** (datasheets, vendor manuals, schematics, technical PDFs, draw.io diagrams) and generates a high-level component model: firmware modules (bootloader, kernel, drivers, OTA updater), communication interfaces (UART / I2C / SPI / ethernet / Wi-Fi), trusted vs. untrusted execution domains, external dependencies (libs, 3rd-party SDKs), and attack surfaces (ports, exposed APIs, file-system mounts). The output is persisted as a **component knowledge graph**: nodes are firmware modules, communication interfaces, execution domains, external dependencies and attack surfaces; edges capture relationships such as *depends-on*, *exposes*, *talks-to*, *is-isolated-from*. The graph lives in the same SQLite / DuckDB store as the structured Inventory tables and is queryable by both the conversational Q&A flow (vector + graph hybrid retrieval) and a graph-view UI element. The graph is incrementally built across all bundle documents, so cross-document hierarchies (e.g. "this firmware module depends on a 3rd-party SDK whose CVE is in the local advisories repo") become explicit. SBOM-style hierarchies and firmware memory maps are extracted in the same pass when documentation provides them. This addresses HMGCC's stated dissatisfaction with vanilla RAG (Q34: "poor handling of structured artifacts, difficulty representing hierarchies such as SBOM graphs, firmware memory maps") and answers Q35 ("Both" structured representation and Q&A) directly. HMGCC test data does not include binary firmware or source code (Q21), so this extraction is documentation-driven by design. If a researcher chooses to ingest source code outside HMGCC test data, tree-sitter parsers handle the same architectural extraction at the file level; deep static analysis remains explicitly out of scope (Q14: code analysis is not a priority).
- **Stage 3, Language detection.** Per-chunk detection using **fastText lid.176**. Triggers translation pipeline for non-English chunks; both original and translated forms are stored, citations always reference the original.
- **Stage 4, ICS-specific extraction.** Protocol-aware extractors run after generic NLP. Each ICS protocol (Modbus, S7Comm, EtherNet/IP, OPC UA, DNP3, IEC 60870-5-104) has a dedicated extractor that recognises function codes, register addresses, object identifiers and protocol-specific terminology, and writes structured records into the Inventory / Ports / Services / Components tables.
- **Stage 5, Embedding and indexing.** Each chunk gets a BGE-M3 dense embedding plus a sparse representation; both are stored in ChromaDB (the existing in-memory vector store is kept as a fast in-process cache). Structured records are written to SQLite / DuckDB.

- **Stage 6, Provenance recording.** Every chunk and every structured record carries an immutable provenance record (source file hash, extraction stage, model versions, timestamp). This is what makes Findings reproducible across the project.

A.11 Multilingual / cross-lingual implementation

Existing baseline (already in production code). OTShield ships an offline output translator today across **5 languages** (English, Turkish, German, French, Spanish), implemented in TranslationService.java as a thin layer over the local Ollama chat model. The system prompt is hand-tuned for the ICS / cyber-security domain: citation markers ([1], [2], [3], etc.), CVE ids, port numbers, IP / MAC addresses, file names, version strings and ICS acronyms (PLC, HMI, ICS, SCADA, Modbus, S7Comm, OPC UA, etc.) are preserved verbatim. Translations are LRU-cached in memory so reopening a thread does not re-translate identical content. Failure modes are non-blocking: on Ollama timeout, error or empty output the original text is returned, so translation is never load-bearing on the user flow. The frontend (translationService.ts, /api/assistant/translate) exposes the language dropdown directly in the Threads tab.

Phase 1 upgrade. The Phase 1 multilingual capability extends this baseline in four concrete sub-systems that map onto the desirable requirement in main proposal Section 3.

- **Dedicated NMT model.** Distilled NLLB-200 weights prioritising HMGCC-stated languages (DE, JA, FR, ZH) are packaged offline and replace the chat-model translator on the languages it supports. Quantised to int8 to fit alongside Gemma 4 in workstation memory. NLLB-200 covers 200+ languages, so additional priorities can be activated without re-packaging. The chat-model translator is retained as a fallback for languages outside the NLLB priority set.
- **Bilingual ICS-term dictionary.** Curated jointly with HMGCC during S3, seeded from open ICS standards (IEC 60870, IEC 61850, ISA-99 / IEC 62443 glossaries) and from OTShield's training corpus. The dictionary acts as a *translation override*: ICS-specific terms are translated through the dictionary, not through generic NLLB, to avoid loss of meaning (e.g. "function code", "coil", "holding register" are precise terms that NLLB alone would mis-translate).
- **Cross-lingual retrieval.** BGE-M3's native multilingual embedding space allows an English query to retrieve relevant German, French, Chinese passages without round-trip translation. Translation is invoked only at *presentation* time, not at retrieval time, which preserves precision.
- **Translation-quality benchmark.** A held-out set of 200 ICS-specific bilingual sentence pairs is used to compute BLEU and chrF++ in Sprint 2. Targets: BLEU ≥ 35 , chrF++ ≥ 60 on the ICS-specific test set, deliberately stricter than generic NLLB benchmarks because the dictionary override raises the floor.

A.12 Bias mitigation (cultural-bias desirable requirement)

This section expands the cultural-bias mitigation summary in main proposal Section 3 desirable (c). The design follows the HMGCC Q&A Q15 framing precisely: a four-layer mechanism with three user-selectable policy modes, integrated with the existing offline MT engine. The bias-flag layer sits on top of the verifier agent's output rather than inside model fine-tuning, because air-gap update bundles cannot retrain the model in place. Aligned with NCSC RTAU (Responsible Technology Adoption Unit) bias-mitigation guidance (rtau.blog.gov.uk/category/bias).

Three policy modes (user-selectable)

Policy	Behaviour
PROACTIVE	Assistant automatically pulls missing alternative-cultural sources from the local repo (or prompts the user to obtain them) and corrects biased statements without asking.

Policy	Behaviour
SUGGESTIVE	Assistant flags potential biases to the user but takes no automatic corrective action.
OFF	Vanilla retrieval; no bias checking. For situations where the corpus is known to be limited and the researcher wants raw output.

Four-layer mechanism with concrete UI flow

Layer	Action	User sees
1. Ingestion	User uploads a folder (e.g. ThermoCo_T200/) containing only English documents. Ingestion engine tags each chunk with lang=EN and origin=NL (vendor HQ jurisdiction). It compares against locally available repositories: "Missing Dutch source → 0/2 Dutch documents available in local repo".	Modal banner at top of chat window: <i>"All uploaded documents for ThermoCo are English. Two Dutch advisories exist locally (see Cultural-Gap tab)." If no local alternative exists, just the message about 100% English for a Dutch manufacturer.</i>
2. Corpus audit	On upload completion, a corpus-level audit produces a coverage summary across all current bundles plus a follow-up checklist : e.g. "add Dutch security advisories", "include Chinese CERT notices".	Cultural-Gap tab with the audit summary and checklist. Each item is actionable (click to mark resolved or to open the source-search panel).
3. Retrieval	Vector search returns sources; the system computes a diversity score per answer. Example: query against ThermoCo returns 1) English NVD entry (CVE-2023-1234), 2) English vendor advisory (ThermoCo-ReleaseNotes), 3) (no Dutch docs) → diversity score = 100% English.	Coverage label appended to every answer: <i>"Coverage: English 90% / Dutch 0%"</i> . The label is clickable and opens the Cultural-Gap tab.
4. Interaction (PROACTIVE)	System looks for missing Dutch documentation in known sources / repos. Finds ThermoCo_T200_NL_Bulletin_2023_02.pdf. The offline MT engine (existing TranslationService for EN/TR/DE/FR/ES today; NLLB-200 distilled-600M prioritising HMGCC's stated languages DE, JA, FR, ZH in Phase 1) translates the relevant paragraph into English and adds it to the response, with a provenance link/alert.	Answer shows the translated paragraph inline, with a provenance link back to the original Dutch source plus an alert that the translation was machine-generated.

Why this design satisfies HMGCC Q15

- **Stage of workflow**: bias mitigation is applied at **all four stages** the bidder asked about (ingestion, retrieval, reasoning, interaction), not just one.
- **User assumption correction**: the PROACTIVE mode automatically corrects biased statements without asking; SUGGESTIVE mode flags them; OFF mode is vanilla. The choice respects researcher autonomy.
- **Concrete example**: the ThermoCo / Dutch flow above is the working implementation of HMGCC's worked example in Q15.
- **Translator integration**: the offline MT engine is the same one that today serves the multilingual

desirable (b); cultural-bias mitigation re-uses that infrastructure without new model deployment cost.

- **Audit and explainability:** every coverage label, every translation, every PROACTIVE auto-pull is logged in the human-readable decision log (per HMGCC Q12) so the researcher can see exactly what the system did and why.

Region of origin is determined per source via a small lookup table built with HMGCC: vendor HQ jurisdiction, forum domain country, publication country. The lookup is open and editable by the researcher; new origins can be added via the same Cultural-Gap tab. This implements the desirable requirement *"recognise and mitigate cultural biases to ensure a nuanced understanding"* in a way that is both auditable and defensible.